

Lab 13 - Ingress Resources and HTTP Routing

Field	Value
Lab ID	Lab 13
Day / Topic	Day 3 Services and Networking
Duration	50 minutes
Namespace	kcna-lab13
Mapping	LO3 A2 K2, K4
Cluster	Single-node kind cluster, Kubernetes v1.30 or later

Read this before you start

An Ingress is a declaration, not an implementation. The object is served by the

API server; the **routing** is performed by a separately installed **ingress*

controller. A stock kind cluster has no ingress controller and no

IngressClass^{**}, so everything you create in Steps 1-7 will be accepted, will look

correct, and will route nothing.

>

That is the central lesson of this lab. This README runs in two phases:

>

| Phase | What you do | Where |

|---|---|---|

| Phase 1 - required | Author the Ingress, verify it as a **declaration**, and then prove **measurably** that this cluster has no controller to execute it | Section 4, Steps 1-8 |

| Phase 2 - optional | Install a pinned, maintained controller into a lab-owned namespace, reach it with `kubect`l port-forward, and exercise every routing rule for real | Appendix C |

>

Phase 1 never shows you a curl transcript of routed traffic, because none is

produced. Phase 2 gives you exact, reproducible commands and the output you should

expect - run them and compare against your own cluster. Appendix C needs **no new*

cluster^{**}: it works on the training cluster you already have.

1. Objective

By the end of this lab you will be able to:

1. Author a `networking.k8s.io/v1` Ingress with host-based and path-based rules over multiple backend Services.
2. Choose correctly between `pathType: Prefix`, `Exact` and `ImplementationSpecific`,

and state the element-boundary rule that makes /catalog not match /catalogue.

3. Explain ingressClassName and the IngressClass object, and recognise the ****silent failure**** when neither is present.

4. Read the tls block and say precisely what the Ingress does and does not do about TLS.

5. Prove whether a cluster has an ingress controller at all, ranking the evidence

correctly: the absence of any IngressClass and the absence of any controller Pod are the reliable checks; a blank ADDRESS column and Events: <none> are ***suggestive but not proof***.

6. ***(Optional, Appendix C)*** Install a pinned ingress controller into a lab-owned

namespace, reach it with kubectl port-forward, and verify each routing rule - including the /catalog vs /catalogue boundary - with real HTTP requests.

2. Prerequisites

- Labs 11 and 12 completed. This lab routes to Service backends, so you must

already be fluent with ClusterIP Services and EndpointSlices. (For completeness: the networking.k8s.io/v1 Ingress API defines ***two*** backend forms - backend.service, used here, and backend.resource, a typed reference to another object. backend.resource is out of scope for KCNA and for this lab, and support for it varies by controller, but "an Ingress can only point at a Service" is not what the API says.)

- A running single-node kind cluster.
- *Optional, for Appendix C only:*** helm (v3 or later) and curl on your workstation.

Appendix C also documents a Helm-free path.

- Working directory:

```
cd courseware/labs/lab-13-ingress-routing
pwd
```

Expected output:

```
/.../courseware/labs/lab-13-ingress-routing
```

- Confirm the Ingress API is served by your cluster:

```
kubectl api-resources --api-group=networking.k8s.io
```

Expected output:

NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
ingressclasses		networking.k8s.io/v1	false	IngressClass
ingresses	ing	networking.k8s.io/v1	true	Ingress
networkpolicies	netpol	networking.k8s.io/v1	true	NetworkPolicy

Note that ingressclasses is cluster-scoped (NAMESPACED false) while ingresses is namespaced. That asymmetry matters in Step 5.

3. Scenario

Meridian Freight's customer portal is being consolidated. Today three services are each exposed on their own NodePort, and partners have to remember three different :3xxx URLs. Worse, Lab 12 showed that node ports come from one cluster-wide pool of 2,768 numbers - that does not scale.

The target design is a single HTTP entry point:

Request	Should reach
http://shop.meridianfreight.internal/catalog and anything below it	catalog-svc
http://shop.meridianfreight.internal/checkout/status - that URL exactly	checkout-svc
http://tracking.meridianfreight.internal/ and anything below it	catalog-svc
anything else	the default backend, route-fallback-svc

You will express that design as an Ingress object, verify the object is correct, and then determine honestly whether this training cluster can execute it.

4. Step-by-step procedure

Step 1 - Namespace and the routing table

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab13 created
```

The intended routing is recorded as data before you write any YAML:

```
cat data/routing-table.csv
```

Expected output:

```
host,path,pathType,service,port
shop.meridianfreight.internal,/catalog,Prefix,catalog-svc,80
shop.meridianfreight.internal,/checkout/status,Exact,checkout-svc,80
tracking.meridianfreight.internal,/,Prefix,catalog-svc,80
```

verification/checks.sh diffs the live Ingress against exactly this file, so the file is the specification and the Ingress is the implementation.

Step 2 - Backends

```
kubectl apply -f manifests/10-configmap-catalog.yaml
kubectl apply -f manifests/20-deployments.yaml
kubectl apply -f manifests/30-services.yaml
kubectl apply -f manifests/35-pod-net-client.yaml
```

Expected output:

```
configmap/catalog-content created
configmap/catalog-nginx created
deployment.apps/catalog created
deployment.apps/checkout created
deployment.apps/route-fallback created
service/catalog-svc created
service/checkout-svc created
service/route-fallback-svc created
pod/net-client created
```

```
kubectl wait --for=condition=Available deployment --all -n kcna-lab13 --timeout=120s
kubectl wait --for=condition=Ready pod/net-client -n kcna-lab13 --timeout=60s
```

Expected output:

```
deployment.apps/catalog condition met
deployment.apps/checkout condition met
deployment.apps/route-fallback condition met
pod/net-client condition met
```

Step 3 - Prove the backends work before involving Ingress

This is the discipline that saves hours later: never debug an Ingress until you have proved the Service behind it answers.

```
kubectl exec -n kcna-lab13 net-client -- wget -q0- --timeout=5 http://catalog-svc/ | head -3
```

Expected output:

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="utf-8"><title>Meridian Freight - Container Catalog</title></head>
```

```
kubectl exec -n kcna-lab13 net-client -- wget -q0- --timeout=5 http://checkout-svc/
```

Expected output:

```
checkout status OK :: kcna-lab13
```

```
kubectl exec -n kcna-lab13 net-client -- wget -q0- --timeout=5 http://route-fallback-svc/
```

Expected output:

```
no ingress rule matched - default backend :: kcna-lab13
```

All three backends are healthy. Remember this: anything that fails from here on is an Ingress-layer problem, not an application problem.

Step 4 - Create the Ingress

```
kubectl apply -f manifests/40-ingress-shop.yaml
```

Expected output:

```
ingress.networking.k8s.io/meridian-shop created
```

```
kubectl get ingress -n kcna-lab13
```

Expected output on a stock kind cluster:

NAME	CLASS	HOSTS
ADDRESS	PORTS	AGE
meridian-shop	meridian-edge	shop.meridianfreight.internal,tracking.meridianfreight.
internal	80	8s

The ADDRESS column is empty, and on this cluster it will stay empty. ADDRESS

is rendered from `.status.loadBalancer.ingress[]`, which only a controller writes.

No controller, no address.

>

But read the implication in the right direction. No controller ☐ blank ADDRESS.

The converse does not hold: blank ADDRESS does **not** prove there is no

controller. A controller that is reachable in-cluster or through a port-forward - the

Appendix C setup is exactly that - routes traffic correctly while publishing no

external address at all, so ADDRESS stays blank. Treat a blank ADDRESS as a

prompt to investigate, and settle the question with `kubectl get ingressclass` and

a controller-Pod search (Step 8).

>

Note also that PORTS shows 80 only - 443 appears once an Ingress in the

namespace has a `tls` block (you will see that change in Step 7).

Step 5 - Read the rules the way a controller would

```
kubectl describe ingress meridian-shop -n kcna-lab13
```

Expected output:

```

Name:          meridian-shop
Labels:        app.kubernetes.io/part-of=meridian-freight
Namespace:     kcna-lab13
Address:
Ingress Class: meridian-edge
Default backend: route-fallback-svc:80 (10.244.0.44:5678)
Rules:
  Host            Path  Backends
  ----
  shop.meridianfreight.internal
                                /catalog      catalog-svc:80 (10.244.0.42:8080)
                                /checkout/status checkout-svc:80 (10.244.0.43:5678)
  tracking.meridianfreight.internal
                                /              catalog-svc:80 (10.244.0.42:8080)
Annotations:    <none>
Events:         <none>

```

Three diagnostics live in this one output - note how strong each one actually is:

Line	Reading	Strength of evidence
Address: empty	Nothing has published a load-balancer address for this Ingress	Weak. Suggestive only. A controller reached via ClusterIP + port-forward, or any controller configured without a published service or ingressEndpoint, routes perfectly and still leaves this blank
Backends show real Pod IPs, e.g. (10.244.0.42:8080)	kubectl resolved each Service to its endpoints. If a backend showed (<error: endpoints "catalog-svc" not found>) the Service name is wrong or has no endpoints - a Lab 11 problem, not an Ingress problem	Strong, about the *backends*
Events: <none>	No controller has written an event on this object	Weak. Some controllers emit Sync-style events, others emit none at all in normal operation. Absence of events is not absence of a controller

The two checks that *do* settle it are in Step 8: is there an IngressClass, and is there a controller Pod.

Verify the object matches the specification in data/routing-table.csv:

```

kubectl get ingress meridian-shop -n kcna-lab13 -o go-template='{{range .spec.rules}}{{ $h := .host }}{{range .http.paths}}{{ $h}}, {{.path}}, {{.pathType}}, {{.backend.service.name}}, {{.backend.service.port.number}}{{ "\n" }}{{end}}{{end}}'

```

Expected output - identical to the CSV body, in order:

```

shop.meridianfreight.internal,/catalog,Prefix,catalog-svc,80
shop.meridianfreight.internal,/checkout/status,Exact,checkout-svc,80
tracking.meridianfreight.internal,/,Prefix,catalog-svc,80

```

Step 6 - pathType, precisely

There are exactly three legal values. Confirm from the live schema rather than memory:

```

kubectl explain ingress.spec.rules.http.paths.pathType

```

Expected output (abridged - the description is long):

```
KIND:      Ingress
VERSION:   networking.k8s.io/v1

FIELD: pathType <string>

DESCRIPTION:
  PathType determines the interpretation of the Path matching. PathType can be
  one of the following values:
  * Exact: Matches the URL path exactly.
  * Prefix: Matches based on a URL path prefix split by '/'. Matching is done on
  a path element by element basis. ...
  * ImplementationSpecific: Interpretation of the Path matching is up to the
  IngressClass. ...
```

Internalise the element-boundary rule, because it is examinable and it is the source of real outages:

Rule	Request path	Match?	Why
/catalog Prefix	/catalog	yes	exact element match
/catalog Prefix	/catalog/	yes	trailing separator
/catalog Prefix	/catalog/containers/20ft	yes	/catalog is a complete leading element sequence
/catalog Prefix	/catalogue	no	catalogue ≠ the element catalog; prefix matching is not string matching
/checkout/status Exact	/checkout/status	yes	identical
/checkout/status Exact	/checkout/status/	no	trailing slash makes it a different path
/checkout/status Exact	/checkout/status?id=7	yes	the query string is not part of the path

When two rules match, the longest matching path wins; where an Exact and a Prefix rule are equally long, Exact takes precedence. ImplementationSpecific hands the decision to the controller - for example, some controllers interpret the path as a regular expression. Portable manifests avoid it.

Step 7 - The TLS block

```
kubectl apply -f manifests/50-ingress-tls-example.yaml
```

Expected output:

```
ingress.networking.k8s.io/meridian-shop-tls created
```

```
kubectl get ingress -n kcna-lab13
```

Expected output - the new object advertises 80, 443:

NAME	CLASS	HOSTS
ADDRESS	PORTS	AGE
meridian-shop	meridian-edge	shop.meridianfreight.internal,tracking.meridianfreight.
internal	80	4m
meridian-shop-tls	meridian-edge	secure.meridianfreight.internal
	80, 443	6s

Look at the structure - this is the entire TLS surface of the Ingress API:

```
kubectl get ingress meridian-shop-tls -n kcna-lab13 -o jsonpath='{.spec.tls}' ; echo
```

Expected output:

```
[{"hosts":["secure.meridianfreight.internal"],"secretName":"meridian-shop-tls-cert"}]
```

Two fields, and that is all:

- hosts - the SNI names this certificate is presented for.
- secretName - a Secret of type `kubernetes.io/tls` in the same namespace,

holding `tls.crt` and `tls.key`.

The Ingress object does not terminate TLS. The controller does, by reading that Secret. Note that the Secret does not exist yet and the object was still accepted - another silent dependency. Create it so you have seen the required shape (a throwaway self-signed certificate, lab use only):

```
openssl req -x509 -nodes -newkey rsa:2048 -days 1 \
  -keyout /tmp/lab13-tls.key -out /tmp/lab13-tls.crt \
  -subj "/CN=secure.meridianfreight.internal" 2>/dev/null
kubectl create secret tls meridian-shop-tls-cert -n kcna-lab13 \
  --cert=/tmp/lab13-tls.crt --key=/tmp/lab13-tls.key
```

Expected output:

```
secret/meridian-shop-tls-cert created
```

```
kubectl get secret meridian-shop-tls-cert -n kcna-lab13
```

Expected output:

NAME	TYPE	DATA	AGE
meridian-shop-tls-cert	kubernetes.io/tls	2	5s

TYPE `kubernetes.io/tls` and DATA 2 (`tls.crt` + `tls.key`) is the exact shape an ingress controller looks for. In production you would never hand-roll this - use `cert-manager` or your organisation's CA.

Step 8 - Determine, honestly, whether this cluster can route

Combine the known cluster configuration, controller inventory and live routing probes. No single empty inventory query proves absence on an arbitrary cluster; controllers may use different names, run externally or accept class-less Ingresses.

Check 1 (inventory) - does any `IngressClass` exist? The object identifies a controller type but does not prove a running controller. Ask which classes exist:

```
kubectl get ingressclass
```

Expected output on a stock kind cluster:

```
No resources found
```

Check 2 (inventory) - is a known controller Pod running? Label conventions differ between projects, so search on the workload name too, not just on one label:

```
kubectl get pods --all-namespaces -l app.kubernetes.io/component=controller
kubectl get pods --all-namespaces -o custom-columns=NS:.metadata.namespace,POD:.metadata.name -
-no-headers \
  | grep -Ei 'ingress|traefik|contour|haproxy|envoy' || echo "no controller-shaped Pod found"
```

Expected output on a stock kind cluster:

```
No resources found
no controller-shaped Pod found
```

Check 3 (corroborating only) - did anything write a status onto your Ingress?

```
kubectl get ingress meridian-shop -n kcna-lab13 -o jsonpath='{.status.loadBalancer}' ; echo
```

Expected output:

```
{}
```

Remember Step 4: an empty `.status.loadBalancer` on its own would not prove

anything. It only corroborates checks 1 and 2. Indeed, when you install the

controller in Appendix C, routing will start working and this field will *still*

print `{}` - because that controller is deliberately configured with a ClusterIP

Service and no published address.

Conclusion, stated plainly: checks 1 and 2 both come back empty, so this cluster has no ingress controller. Your Ingress objects are syntactically correct, semantically correct, and completely inert. No host, port or IP on this cluster will exercise these rules - and if any tutorial shows you a curl transcript without installing a controller first, it is wrong.

An IngressClass is what a controller installs to advertise itself. Here is the shape you would see afterwards, so you can recognise it (this is not created by Steps 1-8; Appendix C creates a real one):

```
apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata:
  name: meridian-edge
  annotations:
    # Optional. It is ONE mechanism by which a controller may pick up an Ingress
    # that names no class. It is not the only one - see section 6.2.
    ingressclass.kubernetes.io/is-default-class: "true"
spec:
  controller: example.com/ingress-controller # the controller's own identifier
```

spec.controller is an opaque identifier that a specific controller binary watches for. Your ingressClassName: meridian-edge is a forward reference to an object that does not exist yet - which is legal, and silent. Appendix C makes that reference resolve by installing a controller that names its IngressClass meridian-edge.

5. Verification

verification/checks.sh has two modes and prints which one it ran:

Mode	Trigger	What it asserts
MODE 1 - declaration-only	no IngressClass exists on the cluster	the Ingress objects are correct *declarations*. Live routing is reported SKIP - never PASS, never FAIL
MODE 2 - controller	an IngressClass exists	everything MODE 1 checks, plus seven real HTTP routing assertions driven through kubectl port-forward

```
bash verification/checks.sh
```

Expected output on a stock kind cluster (MODE 1):

```
== Lab 13 verification: Ingress resources and HTTP routing ==
PASS namespace kcna-lab13 exists
PASS ConfigMap catalog-content matches data/catalog-index.html byte-for-byte
PASS all 3 backend Deployments are Available
PASS catalog-svc, checkout-svc and route-fallback-svc all have endpoints
PASS backend catalog-svc answers directly (Ingress bypassed)
PASS backend checkout-svc answers directly (Ingress bypassed)
PASS backend route-fallback-svc answers directly (Ingress bypassed)
PASS ingress meridian-shop exists with apiVersion networking.k8s.io/v1
PASS ingress rules match data/routing-table.csv exactly
PASS every pathType is one of Exact / Prefix / ImplementationSpecific
PASS ingress meridian-shop sets ingressClassName (meridian-edge)
PASS ingress meridian-shop declares a defaultBackend (route-fallback-svc:80)
PASS ingress meridian-shop-tls declares tls[0].secretName
PASS TLS Secret meridian-shop-tls-cert is type kubernetes.io/tls with 2 keys
NOTE MODE 1 (declaration-only): no IngressClass exists on this cluster.
PASS DECISIVE: 0 IngressClasses exist, so no controller can claim the Ingress
PASS DECISIVE: no Ready controller Pod in namespace kcna-lab13-ingress
NOTE CORROBORATING: .status.loadBalancer is empty (ADDRESS column blank). This does not
establish whether routing works.
SKIP live HTTP routing not exercised: this cluster has no ingress controller (MODE 1)
NOTE The Ingress objects are verified as CORRECT DECLARATIONS only.
NOTE README Appendix C installs a pinned controller and switches this script into MODE 2.
-- 16 passed, 0 failed, 1 skipped --
```

Read the last four lines carefully:

- The blank ADDRESS is now a NOTE, not a PASS. It is corroboration, not proof.

The two DECISIVE lines are what actually settle the question.

- Routing is SKIP, not FAIL. The script did not test it, so it reports neither

success nor failure - that is the honesty contract.

MODE 2 appears once you complete Appendix C. See Appendix C step C6 for the exact expected transcript.

6. Failure injection - an invalid pathType, then a silent one

6.1 Break it - the loud failure

```
kubectl apply -f manifests/60-ingress-invalid-pathtype.yaml
```

Expected output (rejected; non-zero exit):

```
The Ingress "meridian-shop-broken" is invalid: spec.rules[0].http.paths[0].pathType:
Unsupported value: "prefix": supported values: "Exact", "ImplementationSpecific", "Prefix"
```

Diagnosis. pathType is a case-sensitive enum. prefix is not Prefix. The error names the field path (spec.rules[0].http.paths[0].pathType), the rejected value, and the complete legal set - read the whole line before guessing.

Confirm nothing was created:

```
kubectl get ingress meridian-shop-broken -n kcna-lab13
```

Expected output:

```
Error from server (NotFound): ingresses.networking.k8s.io "meridian-shop-broken" not found
```

Note again that the file is schema-valid:

```
kubeconform -strict -summary manifests/60-ingress-invalid-pathtype.yaml
```

Expected output:

```
Summary: 1 resource found parsing 1 file - Valid: 1, Invalid: 0, Errors: 0, Skipped: 0
```

pathType is typed as a plain string in the OpenAPI schema; the enum is enforced by API-server validation. Static linting cannot catch this class of defect.

6.2 Fix defect 1 - and meet the silent failure

```
sed 's/pathType: prefix/pathType: Prefix/' manifests/60-ingress-invalid-pathtype.yaml \
| kubectl apply -f -
```

Expected output:

```
ingress.networking.k8s.io/meridian-shop-broken created
```

It was accepted. Now look at it:

```
kubectl get ingress meridian-shop-broken -n kcna-lab13
```

Expected output - note the empty CLASS column:

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
meridian-shop-broken	<none>	shop.meridianfreight.internal		80	12s

```
kubectl describe ingress meridian-shop-broken -n kcna-lab13 | grep -E 'Ingress
Class|Address|Events'
```

Expected output:

```
Ingress Class:    <none>
Address:
Events:          <none>
```

Diagnosis of the silent failure. CLASS <none> means this Ingress named no class. What happens next is controller-dependent, and this is a distinction worth getting right:

- On a cluster with no controller at all - this cluster, right now - nothing claims

it. Guaranteed silence.

- The ingressclass.kubernetes.io/is-default-class: "true" annotation is one

mechanism by which a controller may adopt a class-less Ingress. It is a common one, but it is not the only one, and it is not a universal rule.

- Some controllers additionally claim class-less Ingresses on their own terms. Traefik,

for instance, documents that when its providers.kubernetesIngress.ingressClass option is empty it processes "resources missing the annotation, having an empty value, or the value traefik" (<https://doc.traefik.io/traefik/reference/install-configuration/providers/kubernetes/kubernetes-ingress/>).

- Other controllers ignore class-less Ingresses outright unless configured otherwise.

The operational rule that survives all of that: name the class explicitly. Never rely on a default, because you cannot predict which controller will or will not pick your object up.

Check whether this cluster has a default IngressClass:

```
kubectl get ingressclass -o jsonpath='{range .items[*]}.{metadata.name}{"\t"}{.metadata.
annotations.ingressclass\.kubernetes\.io/is-default-class}{"\n"}{end}'}
echo "(no output above means: no IngressClass at all)"
```

Expected output:

```
(no output above means: no IngressClass at all)
```

On this cluster there is no controller and no IngressClass, so the object is certainly ignored. On a cluster that *did* have a controller, whether it is adopted depends on that controller's documented class-matching behaviour. A controller may emit events or status; neither is required for successful routing. Test the configured host and path through the controller endpoint. This is the single most common Ingress support ticket:

"I applied the Ingress and nothing happens."

Your diagnostic checklist:

1. kubectl get ingressclass - does any controller advertise itself? *(inventory)*
2. Is a controller Pod running anywhere? *(inventory)*
3. kubectl get ingress -n <ns> - is the CLASS column populated, and does that name match an installed IngressClass?

4. If CLASS is <none>: read your controller's documentation on class-less Ingresses. Do not assume the default-class annotation is the only rule.
5. `kubectl describe ingress` - do the backends resolve to real endpoints?
6. `kubectl describe ingress` - any controller Events? *(weak signal; many controllers emit none)*
7. ADDRESS blank? *(weakest signal; a correctly routing ClusterIP controller leaves it blank forever)*

6.3 Clean up the broken object

```
kubectl delete ingress meridian-shop-broken -n kcna-lab13
```

Expected output:

```
ingress.networking.k8s.io "meridian-shop-broken" deleted
```

7. Troubleshooting

Symptom	Likely cause	Command that confirms it	Fix
ADDRESS column stays empty forever	Often no controller - but not necessarily. A controller with a ClusterIP Service, or one configured without a published service / ingressEndpoint, routes correctly and never fills this in	Inspect the configured controller using <code>kubectl get ingressclass</code> and a controller-Pod search. Events: <none> is only a weak extra hint	If the known stock-kind configuration has no controller: expected on stock kind - install a controller (Appendix C). If a controller <code>*is*</code> present, ignore the blank ADDRESS and test routing directly
Routing works but ADDRESS is still blank	The controller is not publishing a status address (ClusterIP Service, or publishing disabled)	<code>curl</code> through a port-forward succeeds while <code>kubectl get ingress</code> shows no ADDRESS	Nothing to fix. This is the Appendix C configuration by design
CLASS shows <none> and nothing routes, with no error	<code>ingressClassName</code> omitted; whether any controller adopts it is controller-dependent	<code>kubectl get ingressclass -o jsonpath=...is-default-class</code> , plus your controller's own class-matching documentation	Set <code>spec.ingressClassName</code> explicitly. Prefer explicit over default, always
<code>/catalogue</code> unexpectedly reaches <code>catalog-svc</code> on a controller you installed	That controller is doing character-by-character prefix matching instead of the spec's path-element matching	Compare <code>/catalog</code> and <code>/catalogue</code> responses through the port-forward	Turn on the controller's spec-conformance option - for Traefik, <code>providers.kubernetesIngress.strictPrefixMatching=true</code> , already set in <code>data/traefik-lab-values.yaml</code>
Unsupported value: "prefix" on apply	<code>pathType</code> enum is case-sensitive	Read the error - it lists all three legal values	Use Prefix, Exact or ImplementationSpecific
<code>spec.rules[0].http.paths[0].path</code> : Invalid value: "catalog": must be an absolute path	Path is missing its leading /	The API-server error names the exact field path	Write <code>/catalog</code>
<code>describe</code> shows <error: endpoints "catalog-svc" not found> under Backends	The named Service does not exist, or has no endpoints	<code>kubectl get endpointslices -n kcna-lab13</code>	Fix the Service name/selector first - this is a Lab 11 fault surfacing in an Ingress
Requests to <code>/catalogue</code> unexpectedly 404	Prefix matches whole path elements, not strings	Re-read the <code>pathType</code> table in Step 6	Add an explicit rule, or restructure the URL
<code>/checkout/status/</code> 404s while <code>/checkout/status</code> works	Exact does not tolerate a trailing slash	Compare the two requests	Use Prefix, or add a second Exact rule
Controller logs <code>secret ... not found for TLS</code>	The <code>tls[].secretName</code> Secret is missing or is not type <code>kubernetes.io/tls</code>	<code>kubectl get secret <name> -n kcna-lab13 -o jsonpath='{.type}'</code>	Create it with <code>kubectl create secret tls ...</code> , or use <code>cert-manager</code>

8. Cleanup

Delete only this lab's namespace. The Ingress, Services, Deployments, ConfigMaps and the TLS Secret all live inside it.

```
kubectl delete namespace kcna-lab13
```

Expected output:

```
namespace "kcna-lab13" deleted
```

Remove the throwaway certificate files from /tmp:

```
rm -f /tmp/lab13-tls.key /tmp/lab13-tls.crt && echo "temp certificate removed"
```

Expected output:

```
temp certificate removed
```

Steps 1-8 create no cluster-scoped objects at all.

>

If you also did Appendix C, remove the controller - it *does* create

cluster-scoped objects (an IngressClass plus a ClusterRole and ClusterRoleBinding):

>

```
```bash
```

```
helm uninstall meridian-edge -n kcna-lab13-ingress
```

```
kubectl delete namespace kcna-lab13-ingress
```

```
kubectl get ingressclass
```

```
```
```

>

Expected output - helm uninstall removes the IngressClass, ClusterRole and

ClusterRoleBinding it created, so the cluster returns to its stock state:

>

```
```
```

```
release "meridian-edge" uninstalled
```

```
namespace "kcna-lab13-ingress" deleted
```

```
No resources found
```

```
```
```

>

If you installed the vendored manifest instead of using Helm, delete it with

```
kubectl delete -f data/traefik-41.4.0-rendered.yaml before deleting the namespace.
```

>

If you additionally built the optional cluster in Appendix A, delete it

separately with `kind delete cluster --name kcna-ingress`. Appendix C does not

need that cluster.

```
---
```

9. What you learned

- An Ingress is data, not behaviour. The API server stores it; an ingress

controller implements it. No controller means no routing, no error and no event.

- `ingressClassName` binds an Ingress to a controller. Omitting it makes the outcome

controller-dependent: the `is-default-class` annotation is one adoption mechanism among several, some controllers claim class-less Ingresses on their own terms, and others ignore them entirely. Name the class explicitly and the ambiguity disappears.

- `pathType` has exactly three values, is case-sensitive, and Prefix matches whole

path elements - `/catalog` never matches `/catalogue`. Longest match wins; Exact beats Prefix at equal length. Controllers can and do deviate from this, so verify it against the controller you actually run (Appendix C does exactly that).

- `defaultBackend` expresses the unmatched-request fallback. Appendix C supplies a

separate rule-free Ingress because the pinned Traefik version requires that form.

- The `tls` block is only `hosts` + `secretName`, pointing at a

`kubernetes.io/tls` Secret in the same namespace. The controller terminates TLS.

- This lab routes to Service backends (`backend.service`) - so an Ingress fault is

very often a Lab 11 endpoint fault in disguise. Always test the backend Service directly first. The API also defines `backend.resource` for typed object references; it is out of scope here, but "Ingress can only point at a Service" is inaccurate.

- Establish actual routing behaviour: combine the known cluster configuration,

IngressClass/controller inventory, readiness and HTTP probes. Empty inventory queries or blank ADDRESS alone are insufficient; a working controller behind a ClusterIP Service may leave status and events empty.

10. Further reading

- Ingress - <https://kubernetes.io/docs/concepts/services-networking/ingress/>
- Ingress API reference (`backend.service` and `backend.resource`) - <https://kubernetes.io/docs/reference/kubernetes-api/networking/ingress-v1/>
- Ingress Controllers - <https://kubernetes.io/docs/concepts/services-networking/ingress-controllers/>
- `pathType` and path matching - <https://kubernetes.io/docs/concepts/services-networking/ingress/#path-types>
- IngressClass and the default-class annotation - <https://kubernetes.io/docs/concepts/services-networking/ingress/#default-ingress-class>
- TLS in Ingress - <https://kubernetes.io/docs/concepts/services-networking/ingress/#tls>
- ingress-nginx controller project retirement announcement - <https://kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/>
- Gateway API - <https://kubernetes.io/docs/concepts/services-networking/gateway/>
- kind: Ingress setup - <https://kind.sigs.k8s.io/docs/user/ingress/>
- `kubectrl port-forward` - <https://kubernetes.io/docs/tasks/access-application-cluster/port-forward-access-application-cluster/>

- Traefik: getting started on Kubernetes (Appendix C install method) - <https://doc.traefik.io/traefik/getting-started/kubernetes/>
- Traefik: Kubernetes Ingress provider options (strictPrefixMatching, namespaces, publishedService) - <https://doc.traefik.io/traefik/reference/install-configuration/providers/kubernetes/kubernetes-ingress/>
- Traefik Helm chart source and examples - <https://github.com/traefik/traefik-helm-chart>

Appendix A - a kind cluster with host ports (optional, and ***not*** required)

You do not need this appendix to get live routing. Appendix C runs a real controller on the cluster you already have, using `kubectl port-forward`. Read this one only to understand what `extraPortMappings` buys you, and when it is worth recreating a cluster for.

Optional. Creates a second cluster; does not touch your training cluster.

```
cat data/kind-cluster-ingress.yaml
```

Expected output (abridged):

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  kubeadmConfigPatches:
  - |
    kind: InitConfiguration
    nodeRegistration:
      kubeletExtraArgs:
        node-labels: "ingress-ready=true"
  extraPortMappings:
  - containerPort: 80
    hostPort: 80
    listenAddress: "127.0.0.1"
    protocol: TCP
  - containerPort: 443
    hostPort: 443
    listenAddress: "127.0.0.1"
    protocol: TCP
```

```
kind create cluster --name kcna-ingress --config data/kind-cluster-ingress.yaml
```

Both settings are create-time only - a running kind cluster cannot be retrofitted with either:

- `node-labels: ingress-ready=true` - some controller manifests packaged for kind pin the controller Pod to a node carrying this label.
- `extraPortMappings` for 80/443 - publishes the node container's ports on your laptop

so `http://localhost/` reaches the controller on the real, unprefix port.

****What these settings are *not*. They are not the only way to reach a cluster workload from the host, and their absence does not** prevent you from exercising an ingress controller:**

| Access method | Needs cluster recreation? | Reaches the controller on |
|-----------------------------------|-----------------------------------|---|
| kind extraPortMappings + hostPort | Yes - create-time only | http://localhost/ (real port 80/443) |
| kubectl port-forward | No - works on any running cluster | http://127.0.0.1:<local-port>/ |
| NodePort Service | No | the node's :3xxxx port (still needs a host route into the node on kind) |

Appendix C uses kubectl port-forward precisely so that no new cluster is needed. The only thing you give up is the cosmetic convenience of port 80: you send Host: headers to 127.0.0.1:18080 instead of 127.0.0.1:80. The Ingress rules under test are identical.

Appendix B - the controller landscape, stated carefully (checked 5 September 2026)

The networking.k8s.io/v1 Ingress API is stable and not deprecated - everything you wrote in this lab remains correct. What changed is the **controller** landscape.

Two different things called "nginx" - do not conflate them

>

| Thing | Status | Where it appears in these labs |
|--|--------|--------------------------------|
| The Kubernetes community ingress-nginx controller project Retired in March 2026 (announced at https://kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/). Do not select it for a new deployment Nowhere. This lab does not use it | | |
| The NGINX web server container image, nginx:1.27-alpine Unrelated and unaffected. It is an ordinary web server image The catalog backend Pod in this lab, and backends in several other labs | | |

|---|---|---|

| The Kubernetes community ingress-nginx controller project | Retired in March 2026 (announced at <https://kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/>). Do not select it for a new deployment | Nowhere. This lab does not use it |

| The NGINX web server container image, nginx:1.27-alpine | Unrelated and unaffected. It is an ordinary web server image | The catalog backend Pod in this lab, and backends in several other labs |

>

The retirement of a **controller project** says nothing about the **web server image**.

They share a name and nothing else.

What this lab verifies about controllers, and what it does not.

| Claim | Status |
|--|--|
| Traefik chart 41.4.0 / app v3.7.12 installs on a single-node kind cluster and serves networking.k8s.io/v1 Ingress | The path is pinned, reproducible and documented in Appendix C. Run it and confirm on your own cluster |
| Traefik's default prefix matching is character-by-character, and providers.kubernetesIngress.strictPrefixMatching=true makes it comply with the Ingress spec | Documented by the project: https://doc.traefik.io/traefik/reference/install-configuration/providers/kubernetes/kubernetes-ingress/ |
| The community ingress-nginx controller project is retired | Announced by the Kubernetes project (link above) |
| Any <i>*ranking*</i> of which other controllers are "best" or "most maintained" | Not asserted here. Maintenance status changes; check it yourself at install time |

How to evaluate a controller yourself, rather than trusting a table in any courseware:

1. Start from the upstream list -

<https://kubernetes.io/docs/concepts/services-networking/ingress-controllers/> - and then go to each project's own site.

2. Check the release cadence and the latest release date in its own repository.

3. Check that it explicitly supports the networking.k8s.io/v1 Ingress API, not only Gateway API.

4. Read its documentation on pathType conformance and on how it treats an Ingress with no ingressClassName. Both are controller-dependent (sections 6.2 and 7).

5. On a managed cluster, evaluate the provider's own controller first (AWS Load Balancer Controller, GKE Ingress, AGIC and similar), because it integrates with that platform's load balancers.

A precision point about Gateway API. gateway.networking.k8s.io - GatewayClass / Gateway / HTTPRoute - is the newer, role-oriented design that expresses natively much of what Ingress needed vendor annotations for. Note carefully:

- Gateway API is a separate API family, not a new version of Ingress.
- Projects such as Envoy Gateway are Gateway API implementations. Do not

describe them as drop-in Ingress-controller replacements; if you need networking.k8s.io/v1 Ingress, confirm from the project's own documentation whether and how it serves that API before choosing it.

- KCNA expects you to know Ingress, and to know that Gateway API exists and why.

Appendix C - install a real controller and exercise the routes (optional, ~15 min)

Status of the transcripts in this appendix. They are the ****expected result of the**

documented path**, derived from the pinned chart's own rendered manifests and the

project's documentation. They are not a recording of a run on your cluster.

Execute the commands and compare. If your output differs, your output is the truth -

investigate, and treat any mismatch as a defect in this appendix.

What you are about to install

| Property | Value |
|------------------------------|--|
| Controller | Traefik Proxy |
| Helm chart | traefik 41.4.0, repo https://traefik.github.io/charts |
| Chart sha256 | 1470656b1c93e8637daaa19cdb30633c5ea691768a9aab3214da3ee822c4a9f0 |
| App version | v3.7.12 → image docker.io/traefik:v3.7.12 |
| Chart kubeVersion constraint | >=1.25.0-0 |
| Install method | Helm, per the project's own docs:
https://doc.traefik.io/traefik/getting-started/kubernetes/ |
| Namespace | kcna-lab13-ingress - lab-owned, not kube-system |
| Ingress watch scope | kcna-lab13 only |
| Service type | ClusterIP (no LoadBalancer, which would hang <pending> on kind) |
| Host access | kubectl port-forward - no new cluster, no extraPortMappings |
| Traefik CRDs | none installed (--skip-crds + providers.kubernetesCRD=false) |
| Gateway API | disabled |
| IngressClass created | meridian-edge, not marked default |

Every value is set in data/traefik-lab-values.yaml, which is commented line by line.

The cluster-scoped footprint, stated up front

Steps 1-8 create nothing cluster-scoped. This appendix does, because an ingress controller genuinely cannot work without it. Exactly three cluster-scoped objects:

| Object | Why it must be cluster-scoped |
|--|--|
| IngressClass/meridian-edge | IngressClass is a cluster-scoped kind by API design (you confirmed this in section 2: NAMESPACE false). A controller advertises itself with one; there is no namespaced equivalent |
| ClusterRole/meridian-edge-traefik-kcna-lab13-ingress | The controller must read that cluster-scoped IngressClass to know which Ingresses are its own. The chart's own documentation states that namespaced RBAC (rbac.namespaced=true) is incompatible with using IngressClass, which is why this lab leaves it false |
| ClusterRoleBinding/... | Binds the above ClusterRole to the controller's ServiceAccount in kcna-lab13-ingress |

The ClusterRole grants read-only access, plus one write:

| Rule | Verbs | Why the controller needs it |
|---------------------------------|------------------|---|
| "" configmaps, nodes, services | get, list, watch | resolve Service backends and node addresses |
| discovery.k8s.io endpointslices | list, watch | find the Pod endpoints behind each Service (your Lab 11/12 knowledge) |
| "" pods | get | correlate endpoints with Pods |

| Rule | Verbs | Why the controller needs it |
|---|------------------|--|
| "" secrets | get, list, watch | load kubernetes.io/tls Secrets named by spec.tls[].secretName |
| networking.k8s.io ingresses, ingressclasses | get, list, watch | the core job |
| networking.k8s.io ingresses/status | update | the only write verb: publishing ADDRESS. This lab disables publishing anyway |
| "" namespaces | list, watch | discover namespaces to watch |

Everything else - ServiceAccount, Service, Deployment - is namespaced inside kcna-lab13-ingress and disappears with the namespace.

C1 - Preconditions

Complete Steps 1-8 first, so the Ingress objects exist and you have seen the no-controller diagnosis. Then:

```
kubectl config current-context
helm version --short
```

Expected output (your context name will differ - record it, you will not need to change it, because everything below happens on this same cluster):

```
kind-<your-training-cluster>
v3.x.y+g...          # or v4.x.y+g... - both work
```

The chart is apiVersion: v2, so Helm 3 and Helm 4 both install it, and every flag used below (--repo, --version, --skip-crds, --wait, --timeout) exists in both.

Do not switch context. Appendix C deliberately runs on the training cluster you

already have.

C2 - Create the lab-owned controller namespace

```
kubectl apply -f manifests/70-namespace-ingress.yaml
```

Expected output:

```
namespace/kcna-lab13-ingress created
```

C3 - Install the pinned controller

--repo is used instead of helm repo add so nothing in your global Helm configuration is modified.

```
helm install meridian-edge traefik \
  --repo https://traefik.github.io/charts \
  --version 41.4.0 \
  --namespace kcna-lab13-ingress \
  --skip-crds \
  -f data/traefik-lab-values.yaml \
  --wait --timeout 5m
```

Expected output (the LAST DEPLOYED timestamp will differ):

```
NAME: meridian-edge
LAST DEPLOYED: ...
NAMESPACE: kcna-lab13-ingress
STATUS: deployed
REVISION: 1
```

--skip-crds matters: the chart ships traefik.io and hub.traefik.io CRDs that this lab has no use for. Combined with providers.kubernetesCRD: false in the values file, the install adds no CustomResourceDefinitions at all.

<details> <summary>No Helm? Use the vendored manifest instead</summary>

data/traefik-41.4.0-rendered.yaml is the byte-for-byte output of

```
helm template meridian-edge traefik \
  --repo https://traefik.github.io/charts --version 41.4.0 \
  --namespace kcna-lab13-ingress --skip-crds --kube-version 1.31.0 \
  -f data/traefik-lab-values.yaml
```

so it is the same six objects, same pinned image docker.io/traefik:v3.7.12:

```
kubectl apply -f data/traefik-41.4.0-rendered.yaml
```

Expected output:

```
serviceaccount/meridian-edge-traefik created
clusterrole.rbac.authorization.k8s.io/meridian-edge-traefik-kcna-lab13-ingress created
clusterrolebinding.rbac.authorization.k8s.io/meridian-edge-traefik-kcna-lab13-ingress created
service/meridian-edge-traefik created
deployment.apps/meridian-edge-traefik created
ingressclass.networking.k8s.io/meridian-edge created
```

It also passes static validation with no CRD schemas required:

```
kubeconform -strict -summary -kubernetes-version 1.31.0 data/traefik-41.4.0-rendered.yaml
```

```
Summary: 6 resources found in 1 file - Valid: 6, Invalid: 0, Errors: 0, Skipped: 0
```

Uninstall with kubectl delete -f data/traefik-41.4.0-rendered.yaml instead of helm uninstall.

The chart is Apache-2.0 licensed; its LICENSE is vendored as data/TRAEFIK-CHART-LICENSE.txt. </details>

C4 - Readiness: three checks, in order

C4.1 - the Deployment rolled out.

```
kubectl rollout status deployment/meridian-edge-traefik -n kcna-lab13-ingress --timeout=180s
```

Expected output:

```
deployment "meridian-edge-traefik" successfully rolled out
```

C4.2 - the controller Pod is Ready, running the pinned image.

```
kubectl wait --for=condition=Ready pod \
  -l app.kubernetes.io/name=traefik -n kcna-lab13-ingress --timeout=180s
kubectl get pods -n kcna-lab13-ingress
kubectl get deployment meridian-edge-traefik -n kcna-lab13-ingress \
  -o jsonpath='{.spec.template.spec.containers[0].image}' ; echo
```

Expected output (Pod name suffix and age will differ):

```
pod/meridian-edge-traefik-... condition met
NAME                                READY   STATUS    RESTARTS   AGE
meridian-edge-traefik-...          1/1     Running   0           40s
docker.io/traefik:v3.7.12
```

C4.3 - the IngressClass now exists, and your forward reference resolves.

```
kubectl get ingressclass
```

Expected output:

| NAME | CONTROLLER | PARAMETERS | AGE |
|---------------|-------------------------------|------------|-----|
| meridian-edge | traefik.io/ingress-controller | <none> | 45s |

That is the object Step 8 proved was missing. spec.controller is traefik.io/ingress-controller - Traefik's own opaque identifier - and the name meridian-edge is exactly what manifests/40-ingress-shop.yaml has been referencing all along.

Confirm it is not the cluster default, which is what keeps section 6.2's class-less Ingress teaching intact:

```
kubectl get ingressclass meridian-edge \
  -o jsonpath='{.metadata.annotations.ingressclass\.kubernetes\.io/is-default-class}' ; echo
```

Expected output:

```
false
```

C4.4 - and now the point of the whole appendix.

```
kubectl get ingress -n kcna-lab13
```

Expected output - ADDRESS is still blank:

| NAME | ADDRESS | CLASS | PORTS | AGE | HOSTS |
|-------------------|---------|---------------|---------|-----|---|
| meridian-shop | | meridian-edge | 80 | 20m | shop.meridianfreight.internal,tracking.meridianfreight.internal |
| meridian-shop-tls | | meridian-edge | 80, 443 | 16m | secure.meridianfreight.internal |

A controller is installed, running and about to route your traffic correctly - and ADDRESS is blank, because its Service is a ClusterIP and publishedService is disabled, so there is no external address to publish. This is the concrete counter-example to "blank ADDRESS means no controller." Do not let anyone tell you otherwise, including an earlier draft of this lab.

C4.4 - Configure this controller's fallback route

Traefik v3.7.12 creates its default router only for an Ingress whose spec.rules is empty. The defaultBackend declared alongside the rules in meridian-shop does not activate that router in this implementation. Without the additional object below, unmatched requests return Traefik's 404 page not found.

Apply the supplied standalone fallback Ingress before running the routing checks:

```
kubectl apply -f manifests/75-ingress-traefik-fallback.yaml
kubectl get ingress meridian-fallback -n kcna-lab13
```

Expected result: ingress.networking.k8s.io/meridian-fallback created on first application. This object has ingressClassName: meridian-edge, no host/path rules, and defaultBackend.service.name: route-fallback-svc on port 80. Its fallback router handles requests that match none of the more specific routes.

This configuration passed all seven HTTP route checks on the training test cluster. The implementation condition is visible in the [version-pinned Traefik provider source](#).

C5 - Reach the controller from the host and test every rule

Start the port-forward in a second terminal, or in the background:

```
kubectl port-forward -n kcna-lab13-ingress deployment/meridian-edge-traefik 18080:8000 &
PF_PID=$!
sleep 3
```

Expected output:

```
Forwarding from 127.0.0.1:18080 -> 8000
Forwarding from [::1]:18080 -> 8000
```

Why 8000: the container's web endpoint listens on 8000, and the Service maps port 80 → targetPort: web. Forwarding to the Deployment addresses the container port directly. The equivalent through the Service is `kubectl port-forward -n kcna-lab13-ingress svc/meridian-edge-traefik 18080:80`.

Now exercise the routing table. Every request goes to the same 127.0.0.1:18080; only the Host: header and the path change - which is exactly how host-based and path-based Ingress routing works.

```
curl -sS -H 'Host: shop.meridianfreight.internal' http://127.0.0.1:18080/catalog | head -5
```

Expected output:

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="utf-8"><title>Meridian Freight - Container Catalog</title></head>
<body>
<h1>Meridian Freight Pte Ltd</h1>
```

```
curl -sS -H 'Host: shop.meridianfreight.internal' http://127.0.0.1:18080/catalog/containers/
20ft | grep '<h1>'
```

Expected output - Prefix matches the deeper path too:

```
<h1>Meridian Freight Pte Ltd</h1>
```

The element-boundary rule, now measurable:

```
curl -sS -H 'Host: shop.meridianfreight.internal' http://127.0.0.1:18080/catalogue
```

Expected output - /catalogue does not match Prefix: /catalog, so the defaultBackend answers:

```
no ingress rule matched - default backend :: kcna-lab13
```

If you instead get the catalog page here, your controller is doing string-prefix

matching rather than the spec's path-element matching. For Traefik that means

providers.kubernetesIngress.strictPrefixMatching is not set - it is true in

data/traefik-lab-values.yaml for exactly this reason. This is a live demonstration

that controller behaviour must be **verified**, not assumed.

Exact, and its intolerance of a trailing slash:

```
curl -sS -H 'Host: shop.meridianfreight.internal' http://127.0.0.1:18080/checkout/status
curl -sS -H 'Host: shop.meridianfreight.internal' http://127.0.0.1:18080/checkout/status/
```

Expected output:

```
checkout status OK :: kcna-lab13
no ingress rule matched - default backend :: kcna-lab13
```

Host fan-out - a different host, the same Ingress object:

```
curl -sS -H 'Host: tracking.meridianfreight.internal' http://127.0.0.1:18080/ | grep '<h1>'
```

Expected output:

```
<h1>Meridian Freight Pte Ltd</h1>
```

An unmapped host falls through to defaultBackend:

```
curl -sS -H 'Host: unmapped.meridianfreight.internal' http://127.0.0.1:18080/
```

Expected output:

```
no ingress rule matched - default backend :: kcna-lab13
```

Stop the port-forward when you are done experimenting by hand:

```
kill "$PF_PID"
```

C6 - Re-run the verifier: it switches to MODE 2 by itself

verification/checks.sh detects the IngressClass, sets up its own port-forward, and asserts the routes. Stop your manual port-forward first so port 18080 is free.

```
bash verification/checks.sh
```

Expected output (tail):

```
NOTE  MODE 2 (controller present): 1 IngressClass(es) found on this cluster.
PASS  DECISIVE: 1 IngressClass(es) exist, so a controller is installed
PASS  IngressClass 'meridian-edge' exists and names controller 'traefik.io/ingress-controller',
      so meridian-shop is claimable
PASS  DECISIVE: 1 Ready controller Pod(s) in namespace kcna-lab13-ingress
NOTE  CORROBORATING: ADDRESS is blank. That is NOT a failure - a ClusterIP controller reached
      by port-forward publishes no address and still routes.
PASS  port-forward 127.0.0.1:18080 -> kcna-lab13-ingress/meridian-edge-traefik:8000 is up
PASS  ROUTED shop.../catalog (Prefix) -> catalog-svc
PASS  ROUTED shop.../catalog/containers/20ft (Prefix, deeper path) -> catalog-svc
PASS  ROUTED shop.../catalogue does NOT match Prefix /catalog (element boundary) ->
      defaultBackend
PASS  ROUTED shop.../checkout/status (Exact) -> checkout-svc
PASS  ROUTED shop.../checkout/status/ trailing slash breaks Exact -> defaultBackend
PASS  ROUTED tracking.../ (host fan-out, Prefix /) -> catalog-svc
PASS  ROUTED unmapped host -> defaultBackend route-fallback-svc
-- 25 passed, 0 failed, 0 skipped --
```

If your controller lives elsewhere or is named differently, point the script at it:

```
ING_NS=my-ns ING_DEPLOY=my-controller ING_CONTAINER_PORT=8000 LOCAL_PORT=18081 \
bash verification/checks.sh
```

If anything the script needs is missing - curl, the Deployment, a free local port - it reports SKIP for the routing suite and still passes the declaration checks. It never converts "I could not test this" into FAIL.

C7 - Optional: watch the controller-dependent class behaviour for yourself

Section 6.2 said the fate of a class-less Ingress is controller-dependent. With a controller installed you can now test the claim instead of reading about it:

```
sed 's/pathType: prefix/pathType: Prefix/' manifests/60-ingress-invalid-pathtype.yaml |
kubectl apply -f -
kubectl get ingress meridian-shop-broken -n kcna-lab13
kubectl port-forward -n kcna-lab13-ingress deployment/meridian-edge-traefik 18080:8000 &
PF_PID=$!
sleep 3
curl -sS -H 'Host: shop.meridianfreight.internal' http://127.0.0.1:18080/catalog | grep '<h1>'
kill "$PF_PID"
kubectl delete ingress meridian-shop-broken -n kcna-lab13
```

meridian-shop-broken still shows CLASS <none>, and meridian-edge is not the default IngressClass - yet Traefik documents that it processes Ingresses that carry no class annotation when its own ingressClass option is unset. Record what your cluster actually does. Whichever way it goes, the lesson is the same and it is the one the exam and production both reward: *name the class explicitly and this question never arises.*

C8 - Remove the controller

```
helm uninstall meridian-edge -n kcna-lab13-ingress
kubectl delete namespace kcna-lab13-ingress
kubectl get ingressclass
kubectl get clusterrole,clusterrolebinding -o name | grep meridian-edge || echo "no meridian-
edge cluster-scoped objects remain"
```

Expected output:

```
release "meridian-edge" uninstalled  
namespace "kcna-lab13-ingress" deleted  
No resources found  
no meridian-edge cluster-scoped objects remain
```

The cluster is back to its stock state, and bash verification/checks.sh returns to MODE 1. Nothing this appendix created outlives it.